DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ISLAMIC UNIVERSITY OF TECHNOLOGY

A SUBSIDIARY ORGAN OF OIC

LAB REPORT 9

CSE 4512: COMPUTER NETWORKING LAB

Name: Nayeemul Hasan Prince

Student ID: 220041125

Section: 1A

Semester: 5th

Date of Submission: 04.03.2026

---

1. Lab Task

The objective of this lab was to design and implement a simple multi-user TCP chat application named **ChatIUT** using Python socket programming and threading.

The tasks included:

- **Developing a TCP chat server (`tcp_chat_server.py`) that:** Binds to 0.0.0.0 on port 6006, accepts multiple client connections, broadcasts received messages to all other connected clients, handles disconnections gracefully, and logs activities to `server.log`.
- **Developing a TCP chat client (`tcp_chat_client.py`) that:** Accepts server IP and port from the command line, uses two threads (send and receive), and allows a clean exit using `/quit` or Ctrl+C.
- **Testing:** Verifying multi-client communication and server logging.

---

2. Procedure

Step 1: Environment Setup Python 3.x was used to create the two primary scripts: `tcp_chat_server.py` and `tcp_chat_client.py`.

Step 2: Server Implementation

- Created a TCP socket using `socket.AF_INET` and `SOCK_STREAM`.
- Bound the server to `0.0.0.0:6006`.
- Used a global client list protected by a threading lock to manage connections safely.
- Spawned a new `threading.Thread` with `daemon=True` for each client.
- Implemented a broadcast function and logged all events (startup, connect, disconnect, forwarding) to `server.log`.

Step 3: Client Implementation

- Parsed server IP and port using `argparse` and connected to the server.
- Created two threads: a **Sender thread** (stdin to server) and a **Receiver thread** (server to terminal).
- Implemented the `/quit` command for clean disconnection.

---

3. Observations & Results

Observation 1: Multi-client Broadcast When Client A sent a message, Client B and Client C successfully received it. This confirmed that the server was correctly identifying all active connections and forwarding data to everyone except the sender.

```
(.venv) PS E:\chat server> python tcp_chat_client.py 127.0.0.1 6006
Connected to TCP server on 127.0.0.1:6006...
Type messages and press Enter. Use /quit to exit.
Hello everyone
```

```
(.venv) PS E:\chat server> python tcp_chat_server.py
TCP server listening on 0.0.0.0:6006...
Received from ('127.0.0.1', 50904): Hello everyone
```

```
(.venv) PS E:\chat server> python tcp_chat_client.py 127.0.0.1 6006
Connected to TCP server on 127.0.0.1:6006...
Type messages and press Enter. Use /quit to exit.

Received: Hello everyone
```

```
(.venv) PS E:\chat server> python tcp_chat_client.py 127.0.0.1 6006
Connected to TCP server on 127.0.0.1:6006...
Type messages and press Enter. Use /quit to exit.

Received: Hello everyone
```

```
(.venv) PS E:\chat server> python tcp_chat_client.py 127.0.0.1 6006
Connected to TCP server on 127.0.0.1:6006...
Type messages and press Enter. Use /quit to exit.

Received: Hello everyone
Hi
```

```
(.venv) PS E:\chat server> python tcp_chat_server.py
TCP server listening on 0.0.0.0:6006...
Received from ('127.0.0.1', 50904): Hello everyone
Received from ('127.0.0.1', 55985): Hi
```

```
(.venv) PS E:\chat server> python tcp_chat_client.py 127.0.0.1 6006
Connected to TCP server on 127.0.0.1:6006...
Type messages and press Enter. Use /quit to exit.
Hello everyone

Received: Hi
```

Figure 1: Successful Multi-Client Chat Session

Observation 2: Graceful Disconnect When a client exited using the `/quit` command, the server removed the client from its active list without crashing or producing "broken pipe" errors.

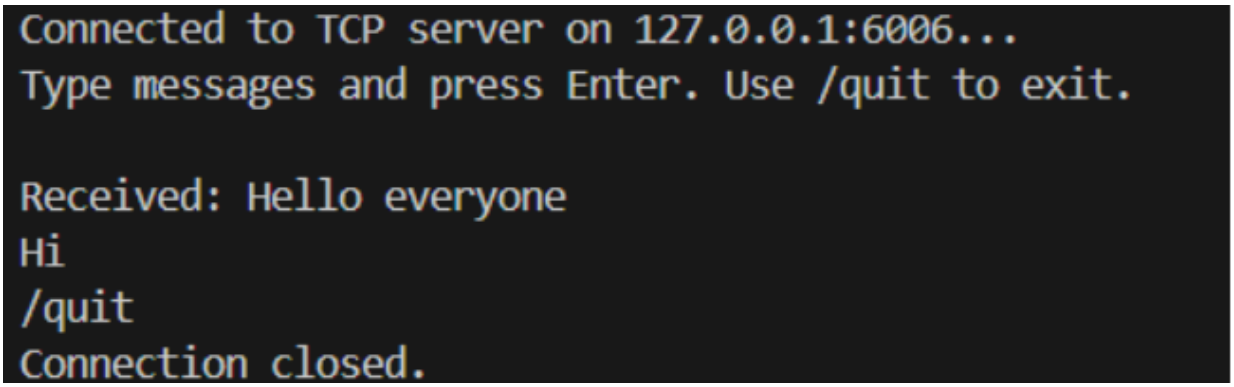

Figure 2: Client disconnect

Observation 3: Logging The `server.log` file correctly recorded the server startup, individual client connections, message forwarding events, and client disconnections with accurate timestamps.

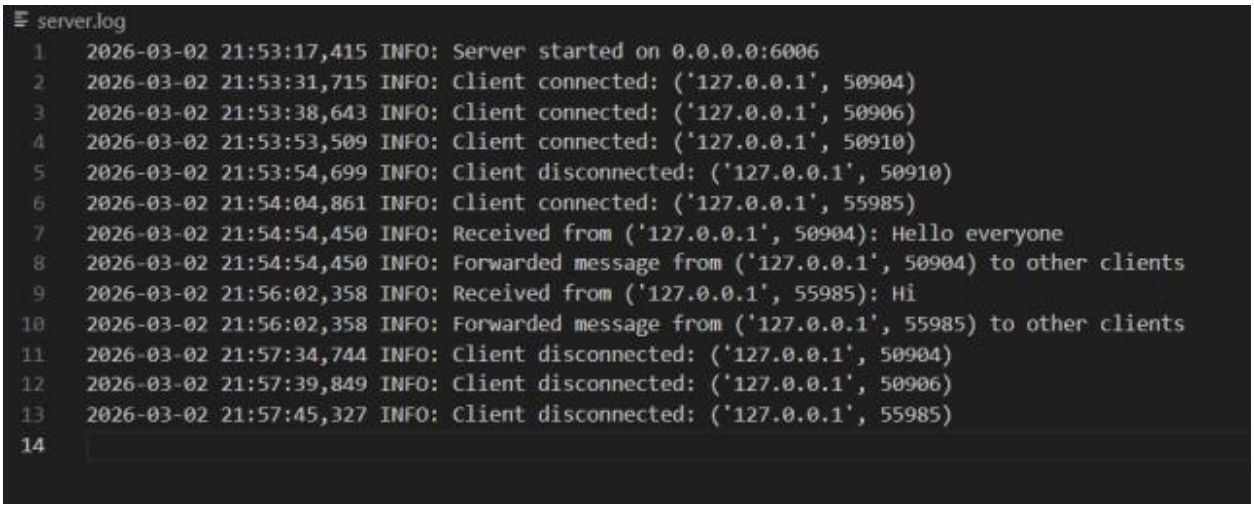

Figure 3: Screenshot of server.log showing startup, message forwarding, and disconnect events

---

4. Challenges Faced

- **Shared List Safety:** Managing the shared client list safely required the proper use of `threading.Lock` to avoid race conditions.
- **Abrupt Terminations:** Handling abrupt client terminations required robust exception handling and cleanup logic to keep the server running.

- **Synchronization:** Ensuring a clean shutdown using daemon threads and avoiding errors during broadcast required the constant removal of "dead" clients from the server's list.

---

5. Conclusion

The **ChatIUT** multi-user TCP chat system was successfully implemented. The server effectively handled multiple concurrent clients using threading, broadcasted messages accurately, and managed both logging and graceful disconnections as required.